

MoneyMind Backend — Архитектура и обоснование решений

Хакатон Клерк.Ру — 29-31 мая 2026

Команда MoneyMind

Содержание

1. Почему Go?	3
2. Почему микросервисы, а не монолит?	3
3. Почему PostgreSQL + ClickHouse?	4
4. Почему Kafka?	5
5. Почему Chi?	6
6. Каждый сервис — почему существует	7
7. Каждый провайдер — почему выбран	9
8. Каждый паттерн — почему так	10
9. Каждый файл — почему существует	12
10. Почему DEMO_MODE	14
11. Почему такие порты	14
12. Что не вошло в MVP и почему	15

1. Почему Go?

Go, не Python, не Java, не Node.js. Go выбран по трём причинам:

Одна бинарка — `go build` даёт исполняемый файл без зависимостей. Для хакатона это критично: мы подняли 15 микросервисов в Docker за 5 минут. В Python нужен виртуальный env со 100+ пакетами, в Node — `node_modules` на 300 МБ.

Горутины — каждый провайдер чеков (ФНС, X5 Club, Магнит) работает в своей горутине. В Go это встроено в язык, не нужен Asyncio/Twisted/Event Loop.

Типизация — JSON чека проходит через 4 сервиса (`scraper` → `Kafka` → `receipt-service` → `Kafka` → `ai-processor`). Если бы не компилятор, расхождение полей мы бы ловили в рантайме на демо.

Почему не Rust? Хакатон — скорость разработки важнее скорости выполнения. Go собирается за 2 секунды, Rust — за 2 минуты.

2. Почему микросервисы, а не монолит?

Микросервисы, монолит — выбрали микросервисы. MVP можно было сделать монолитом в 3 раза быстрее. Но:

Параллельная разработка: фронтенд пишет `/dashboard/*`, тимлид пишет `/chat`, я пишу `/fns/*`. С монолитом мы бы мержили конфликты в одном `routes.go` каждый час.

Независимый деплой: `ai-processor` может упасть (ClickHouse не поднят), но `scraper-service` продолжает качать чеки. В монолите одно падение валит всё.

Изоляция данных: `receipt-service` работает с PostgreSQL, `ai-processor` — с ClickHouse. У них разные профили нагрузки. ClickHouse любит батчи по 1000 строк, PostgreSQL — точечные запросы. В одном процессе они конкурируют за ресурсы.

15 сервисов — много? Для продакшна — да. Для хакатона — это каркас. 9 из 15 — заглушки (просто `NewRouter()` + `/health`). Мы реализовали только 6 ключевых: `api-gateway`, `user-service`, `receipt-service`, `scraper-service`, `ai-processor`, `analytics-service`. Остальные — "кнопки" под будущие фичи.

3. Почему PostgreSQL + ClickHouse?

Постгрес — для фактов, КХ — для аналитики. Это классическая архитектура HTAP (Hybrid Transactional/Analytical Processing), и она здесь оправдана:

PostgreSQL — каждая строка чека — это транзакция. Мы делаем `INSERT INTO receipts ... ON CONFLICT DO NOTHING`. Нам нужны Foreign keys (`user_id` → `users`), `ON CONFLICT` для дедупликации, JSONB для хранения `items` в `receipts`.

ClickHouse — мы строим агрегаты: `SELECT sum(price*quantity) FROM receipt_items WHERE category = 'Продукты'`. Это analytical workload: сканирование миллионов строк, группировка, материализованные представления.

Почему не только ClickHouse? ClickHouse не умеет ON CONFLICT для дедупликации, RETURNING id после вставки, Foreign keys, точечные UPDATE — только мутации с накладными расходами.

Почему не только PostgreSQL? `SELECT category, sum(price*quantity) FROM receipts_items GROUP BY category` — на 10 млн чеков ClickHouse сделает за 50 мс, PostgreSQL — за 3 секунды (даже с индексом). ClickHouse в 100-200 раз быстрее на агрегатах.

Данные дублируются? Да, сознательно. receipt таблица в PG — источник правды. receipt_items в CH — read-optimized копия для аналитики. Это trade-off: +100 мс на запись, -3 секунды на каждый запрос дашборда.

4. Почему Kafka?

Выбор брокера — Kafka, и вот почему:

Лог с retention — если ai-processor упал на час, Kafka хранит сообщения (настраиваемый retention). При перезапуске он дочитает с того же места (consumer group + offset). RabbitMQ удалит сообщение после первого ACK. Redis Streams хранит, но не имеет встроенного механизма переигрывания с контролем offset.

Партиционирование по ключу — receipt.raw партиционируется по receipt.ID. Kafka гарантирует порядок сообщений внутри партии. Один чек всегда обрабатывается последовательно.

Гарантия доставки — Kafka требует подтверждения от всех in-sync replicas. Мы используем RequiredAcks: RequireOne.

Почему не NATS? NATS хорош для микросервисного общения (RPC-style), но не умеет хранить историю. Нам нужна воспроизводимость: если мы добавим новое правило категоризации, мы перечитаем receipt.parsed с начала (reset consumer offset) и перекатегоризируем все чеки. NATS — at-most-once, Kafka — at-least-once.

Почему не RabbitMQ? Rabbit отлично справляется с очередями задач, но плохо — с брокерированием событий. Kafka спроектирован как commit log — то, что нам нужно для event sourcing чеков. RabbitMQ не даёт перечитать старые сообщения без дополнительного кода.

5. Почему Chi?

Chi — самый "стандартный" роутер.

Совместимость с net/http — chi реализует стандартный http.Handler interface. Gin и Fiber — нет. Это значит, что мы можем использовать любую middleware, написанную под стандартную библиотеку.

Встроенный recoverer + CORS — chi.v5/middleware и chi/cors дают всё нужное из коробки.

Нет магии — Fiber копирует Fasthttp — другой API, другой lifecycle. На хакатоне не время разбираться с нюансами.

Контекст — chi.RouteContext передаётся через стандартный context.Context, а не через gin.Context. Это важно для JWT middleware: мы парсим токен и кладем user_id в контекст — и любой http.Handler его читает.

6. Каждый сервис — почему существует

api-gateway (порт 8000)

Единственная точка входа. Фронтенд не должен знать, что у нас 15 сервисов. Он стучится на localhost:8000, gateway рулит.

Почему отдельный сервис, а не nginx? JWT валидация — бизнес-логика. nginx не умеет проверять HMAC-SHA256 JWT без сторонних модулей.

Почему reverse проху, а не gRPC? Фронтенд шлёт JSON. Если бы мы завернули всё в gRPC, пришлось бы ставить grpc-gateway или envoy. На хакатоне это оверхед.

user-service (порт 8001)

Регистрация + JWT. Выделен, чтобы gateway мог доверять токенам. Если бы JWT выпускал scraper-service, это нарушило бы разделение ответственности.

scraper-service (порт 8003)

Самый жирный сервис. Содержит 7 провайдеров: mock, FNS QR, FNS MCO, X5 Club, Magnit, Email IMAP, Email OAuth.

Почему все провайдеры в одном сервисе? Общая инфраструктура: Kafka producer, rate limiter, credential store. Общие типы: RawReceipt, RawItem. Если бы каждый провайдер был отдельным сервисом — 7 × Dockerfile, 7 × main.go, 7 × портов. Для хакатона это оверхед.

Почему не вынести FNS в отдельный сервис? FNS — самый ценный источник. Если он упадёт, остальные провайдеры должны продолжать работать. Но на MVP scraper-service — один процесс. В продакшне FNS можно вынести.

receipt-service (порт 8002)

Pipeline: валидация → дедупликация → сохранение → отправка на категоризацию.

Почему не объединить с scraper-service? Разный lifecycle: scraper может рестартовать для обновления провайдеров, receipt-service — независимо. receipt-service — единственный, кто пишет в receipts таблицу. Единая точка сохранения. Дедупликация требует БД — scraper-service не должен иметь доступ к PostgreSQL (безопасность).

ai-processor (порт 8100)

AI-слой. Читает receipt.parsed, категоризирует (сейчас — словарь, позже — OnlySQL/LLM), пишет в ClickHouse.

Почему не совместить с receipt-service? ClickHouse — аналитическая БД, PostgreSQL — транзакционная. Они должны жить в разных процессах, чтобы не конкурировать за ресурсы. Категоризатор — изолируемый модуль. Замена DictCategorizer на OnlySQLCategorizer — одна строка в main.go.

Почему HTTP для manual expenses? Пользователь вводит расход — ответ должен прийти немедленно. Kafka для этого не подходит (латентность ~50 мс на партицию + обработку).

analytics-service (порт 8101)

Заглушка под инсайты и прогнозы. Выделен, чтобы отделить аналитические отчёты от core pipeline. Если аналитика упадёт, чеки продолжают сохраняться.

Credit, Budget, Category, Goal, Bank services (порты 8004-8011)

Заглушки. Каждая — 20 строк. На MVP не успеваем, но каркас готов: добавь ручку — и сервис работает. Без них пришлось бы втыкать логику в gateway.

7. Каждый провайдер — почему выбран

FNS «Мои Чеки» (mco.nalog.ru) — главный источник

Почему это primary? ФНС — государственный сервис с 2018 года собирает все чеки с онлайн-касс. Если чек пробит на кассе — он есть в базе ФНС. Пользователь регистрирует телефон в приложении «Мои Чеки» — и все его чеки автоматически приходят туда.

Почему Go библиотека jfk9w-go/lkdr-api? Мы не пишем HTTP-клиент с нуля — готовая библиотека с 118 коммитами, которая пережила 53 релиза. Она сама обновляет токены (refresh token), сама решает капчу (через RuCaptcha), сама пагинирует чеки.

Почему RuCaptcha? ФНС поставил reCAPTCHA на страницу входа. RuCaptcha — самый дешёвый (~30 копеек/решение) и стабильный сервис. Альтернатива — 2Captcha (дороже) или AntiCaptcha (та же цена). Решение капчи происходит 1 раз при первой авторизации — токен живёт неделями.

FNS QR-чеки (proverkacheka.nalog.ru) — fallback

Если чек не привязан к телефону (пользователь не регистрировал номер в приложении ФНС) — данные есть только в QR-коде на бумажном чеке. Человек сканирует QR → получает FN, FD, FP → дёргает API proverkacheka.nalog.ru. Это открытое API ФНС — бесплатно, без регистрации и ключей.

Mock-провайдер

Чтобы разработка не блокировалась отсутствием реальных данных. На хакатоне ClickHouse может быть пуст — mock даёт 2 чека, которые проходят весь pipeline до дашборда. Мы отлаживали полный цикл ещё до того, как подключили FNS.

X5 Club и Магнит — запасные источники

Было в изначальном плане. X5 Club и Магнит — крупнейшие ритейлеры с программами лояльности. У них есть API для получения истории покупок. Но оба стоят за Cloudflare — без решения капчи и обхода защиты API не работает стабильно. Решение: закрываем моками. Когда появится время + коммерческая необходимость — включим.

8. Каждый паттерн — почему так

Provider Interface

Единый контракт для всех источников чеков: `type Provider interface { Name(), SendCode(), Login(), Sync() }`. Scheduler не знает, кто он — mock, ФНС или X5 Club. Ему нужен Provider. Добавить новый источник — реализовать 4 метода. `SendCode` + `Login` а не просто `Auth(token)` — потому что разные провайдеры авторизуются по-разному: ФНС МСО — SMS + код, X5 Club — SMS + код, Email — OAuth2.

Categorizer Interface

To replace. Сейчас — DictCategorizer (словарь). Завтра — OnlySQLCategorizer (LLM). Замена — одна строка в `main.go`. Почему не LLM сразу? LLM — 200-500 мс на запрос, 50-100 чеков x 10 позиций = 50-100 секунд на синхронизацию. Словарь — 50 мкс на позицию = 5 мс на весь чек. Для MVP скорость важнее точности. LLM подключаем в фазе 2.

Dedup (SHA-256)

`Hash = SHA256(receiptID + "|" + provider + "|" + userID)`. `receiptID` уже включает FN+FD (`ID = "fns-{FN}-{FD}"`). Дедупликация по ID + провайдер + пользователь — самый надёжный способ: два одинаковых чека от разных провайдеров (FNS МСО и FNS QR) дают одинаковый хеш.

Почему in-memory cache + DB? In-memory cache (`sync.Map`) — 0-1 мс на проверку. Если бы каждый чек проверялся только через PostgreSQL — 10-50 мс на чек x 500 чеков =

5-25 секунд только на дедуп. Cache живёт 7 дней (столько же, сколько TTL в БД). 7 дней — запас: чеки синхронизируются раз в 6-24 часа.

ClickHouse Materialized Views

Материализованное представление считает сумму при вставке, а не при чтении. Дашборд получает ответ за 5-10 мс вместо 1-2 секунд. SummingMergeTree схлопывает строки с одинаковыми ключами при мерже — отклонение в 0.01% допустимо для графиков.

9. Каждый файл — почему существует

Файл	Зачем
go.mod	Единый module. Все сервисы в монорепе — общие зависимости. Не надо обновлять go.mod в 15 папках
internal/router.go	15 сервисов не должны каждый писать один и тот же /health, recoverer, CORS, JSON-логгер. Вынесено в shared
Makefile	make build, make test, make migrate — 3 команды на всё. Не надо помнить порты и пути
.env.example	Единственный источник правды по переменным окружения. Никто не гадает, как настроить ClickHouse

scraper-service/

Файл	Зачем
main.go	Оркестратор: поднимает 7 провайдеров, регистрирует 17 HTTP ручек, запускает Kafka producer и scheduler
internal/types.go	RawReceipt и RawItem — единый формат, в котором данные путешествуют по всему pipeline. Отдельный файл, чтобы не было циклических импортов
internal/provider/interface.go	Provider contract. Позволяет mock и FNS быть взаимозаменяемыми
internal/scheduler/scheduler.go	Периодический сунс провайдеров. Go-ticker — встроенный механизм, не нужен cron
internal/kafka/producer.go	Kafka writer. SendBatch — ключевой метод: 50 чеков = 1 Kafka message вместо 50
internal/ratelimit/limiter.go	Каждый провайдер имеет rate limit. Token Bucket на golang.org/x/time/rate
internal/credentials/crypto.go	AES-256-GCM для хранения

	паролей/токенов провайдеров
internal/providers/mock/provider.go	Демо-режим без внешних зависимостей. 57 строк
internal/providers/fns/handler.go	217 строк: QR-парсинг, CheckTicket (proverkacheka.nalog.ru)
internal/providers/fns_mco/provider.go	333 строки — самый тяжёлый файл. lkdr-арі + авторизация + хранение токенов + синхронизация
internal/providers/x5club/*.go	3 файла: client, mapper, worker
internal/providers/magnit/*.go	3 файла: client, mapper, inn
internal/providers/email/*.go	4 файла: oauth, imap, parser

receipt-service/

Файл	Зачем
main.go	117 строк — самый короткий main. Pipeline: validate → dedup → insert → produce
internal/validator.go	69 строк проверок. Вынесен, чтобы тестировать отдельно
internal/dedup.go	80 строк. Кеш + БД-слой + eviction
internal/repository.go	112 строк CRUD для receipts. Repository pattern
internal/dashboard/handler.go	537 строк — самый большой файл. 5 эндпоинтов + ClickHouse-запросы

ai-processor/

Файл	Зачем
main.go	126 строк. Kafka consumer + HTTP server. Graceful shutdown
internal/types.go	CategorizedReceipt и CategorizedItem — новый тип на выходе категоризатора
internal/categorizer/categorizer.go	Интерфейс из 7 строк. Для замены Dict → OnlySQ
internal/categorizer/dict.go	159 строк. 100+ товаров → 9 категорий. Hardcode
internal/clickhouse/writer.go	88 строк. ClickHouse native protocol — в 10 раз быстрее HTTP
internal/kafka/consumer.go	57 строк. Обёртка над segmentio/kafka-go
internal/manual/handler.go	110 строк. HTTP handler для ручного ввода
internal/manual/repo.go	64 строки. Dual-write: PostgreSQL + ClickHouse
internal/parser/parser.go	112 строк. Регексы — 60+ слов-триггеров

10. Почему DEMO_MODE

DEMO_MODE=true — архитектурное решение: не требует PostgreSQL (работает с in-memory storage), не требует ClickHouse (пишет в /dev/null, дашборд возвращает hardcoded JSON), не требует Kafka (мок-провайдер не шлёт сообщения). Разработчик git clone; cp .env.example .env; make run — и через 5 секунд видит работающий дашборд. Без Docker, без БД, без установки чего-либо.

11. Почему такие порты

8000-8102 — диапазон выбран не случайно: 8000 — gateway (стандартный порт для API-шлюзов), 8001-8011 — бизнес-сервисы (замные с основными системами), 8100-8102 — AI/аналитика (надстройка над бизнес-слоем). Это группировка: если порт начинается на 80 — это core, на 81 — это intelligence.

12. Что не вошло в MVP и почему

Что не вошло	Почему
Банковские API	Условия хакатона (Клерк.Ру): банки не обязательны
X5 Club / Магнит	Cloudflare — без кастомного решения капчи не работает стабильно
OnlySQ / LLM	Для MVP скорость (50 мкс/позицию) важнее точности (98% vs 95%)
Real-time уведомления	WebSocket — отдельный сервис notification-service. Чистое демо можно без них
Система достижений	gamification — пустой. На демо не показываем
Докеризация всех сервисов	9 из 15 — заглушки. Докеризированы для единообразия, но не используются